

Assignment 5: Data Visualization

Humayra Rahman

Spring 2026

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Visualization

Directions

1. Rename this file `<FirstLast>_A05_DataVisualization.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure your code is tidy; use line breaks to ensure your code fits in the knitted output.
5. Be sure to **answer the questions** in this assignment document.
6. When you have completed the assignment, **Knit** the text and code into a single PDF file.

Set up your session

1. Set up your session. Load the tidyverse, here & cowplot packages, and verify your home directory. Read in the NTL-LTER processed data files for nutrients and chemistry/physics for Peter and Paul Lakes (use the tidy NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv version in the Processed_KEY folder) and the processed data file for the Niwot Ridge litter dataset (use the NEON_NIWO_Litter_mass_trap_Processed.csv version, again from the Processed_KEY folder).
2. Make sure R is reading dates as date format; if not change the format to date.

```
#1 loading packages
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(here)
```

```
## here() starts at /home/guest/EDE_Spring2026
```

```
library(cowplot)
```

```
##
```

```
## Attaching package: 'cowplot'
```

```
##
```

```
## The following object is masked from 'package:lubridate':
```

```
##
```

```
## stamp
```

```
getwd()
```

```
## [1] "/home/guest/EDE_Spring2026"
```

```
#2
```

```
lake_nutrients <- read.csv(  
here("Data/Processed_KEY/NTL-LTER_Lake_Chemistry_Nutrients_PeterPaul_Processed.csv"),  
stringsAsFactors = TRUE)
```

```
class(lake_nutrients$sampleddate)
```

```
## [1] "factor"
```

```
litter <- read.csv(  
here("Data/Processed_KEY/NEON_NIWO_Litter_mass_trap_Processed.csv"),  
stringsAsFactors = TRUE)
```

```
class(litter$collectDate)
```

```
## [1] "factor"
```

Define your theme

3. Build a theme and set it as your default theme. Customize the look of at least two of the following:

- Plot background
- Plot title
- Axis labels
- Axis ticks/gridlines
- Legend

```
#3
```

```
#loading packages
```

```
library(ggplot2)
```

```
library(ggthemes)
```

```
library(ggthemes)
```

```
##
## Attaching package: 'ggthemes'

## The following object is masked from 'package:cowplot':
##
##   theme_map

library(viridis)

## Loading required package: viridisLite

library(here)

dir.create(here('output'), showWarnings = FALSE)

#defining a custom theme
my_theme <- theme_grey(base_size = 14) +
  theme(

#customizing plot background
plot.background = element_rect(fill = "lightblue", color = NA),

#customizing plot title
plot.title = element_text(size = 18, face = "bold",
                           color = "#3B4C58", hjust = 0.5),

#customizing axis labels
axis.title.x = element_text(size = 14, face = "bold", color = "#3B4C58"),
axis.title.y = element_text(size = 14, face = "bold", color = "#3B4C58"),

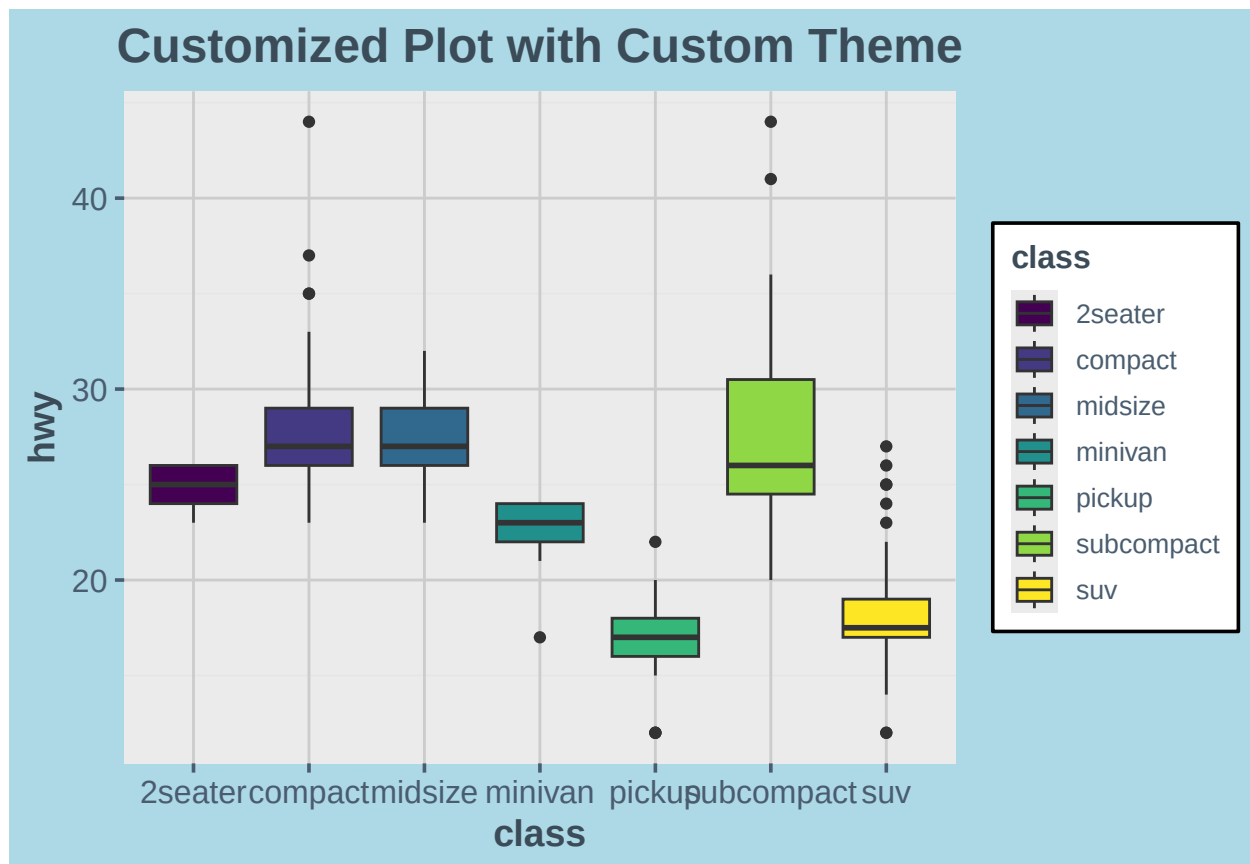
#customizing axis ticks & gridlines
axis.text.x = element_text(size = 12, color = "#4A5D6F"),
axis.text.y = element_text(size = 12, color = "#4A5D6F"),
axis.ticks = element_line(color = "#4A5D6F"),
panel.grid.major = element_line(color = "gray80", linewidth = 0.5),
panel.grid.minor = element_line(color = "gray90", linewidth = 0.25),

#customizing legend
legend.background = element_rect(fill = "white", color = "black"),
legend.title = element_text(size = 12, face = "bold", color = "#3B4C58"),
legend.text = element_text(size = 10, color = "#4A5D6F"),

#removing panel borders
panel.border = element_blank()
  )

#setting the custom theme as default
theme_set(my_theme)

#testing the theme with a simple plot
ggplot(mpg, aes(x = class, y = hwy, fill = class)) +
  geom_boxplot() +
  ggtitle("Customized Plot with Custom Theme") +
  scale_fill_viridis_d()
```



Create graphs

For numbers 4-7, create ggplot graphs and adjust aesthetics to follow best practices for data visualization. Ensure your theme, color palettes, axes, and additional aesthetics are edited accordingly.

4. [NTL-LTER] Plot total phosphorus (tp_ug) by phosphate (po4), with separate aesthetics for Peter and Paul lakes. Add line(s) of best fit using the `lm` method. Adjust your axes to hide extreme values (hint: change the limits using `xlim()` and/or `ylim()`).

```
#4

library(ggplot2)
library(dplyr)

# plotting total phosphorus by phosphate + separating by lake
ggplot(lake_nutrients, aes(x = po4, y = tp_ug, color = lakename)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  xlim(0, 50) +
  ylim(0, 150) +
  labs(
    title = "Total Phosphorus vs Phosphate",
    x = "Phosphate (P04, µg/L)",
    y = "Total Phosphorus (TP, µg/L)",
  )
```

```

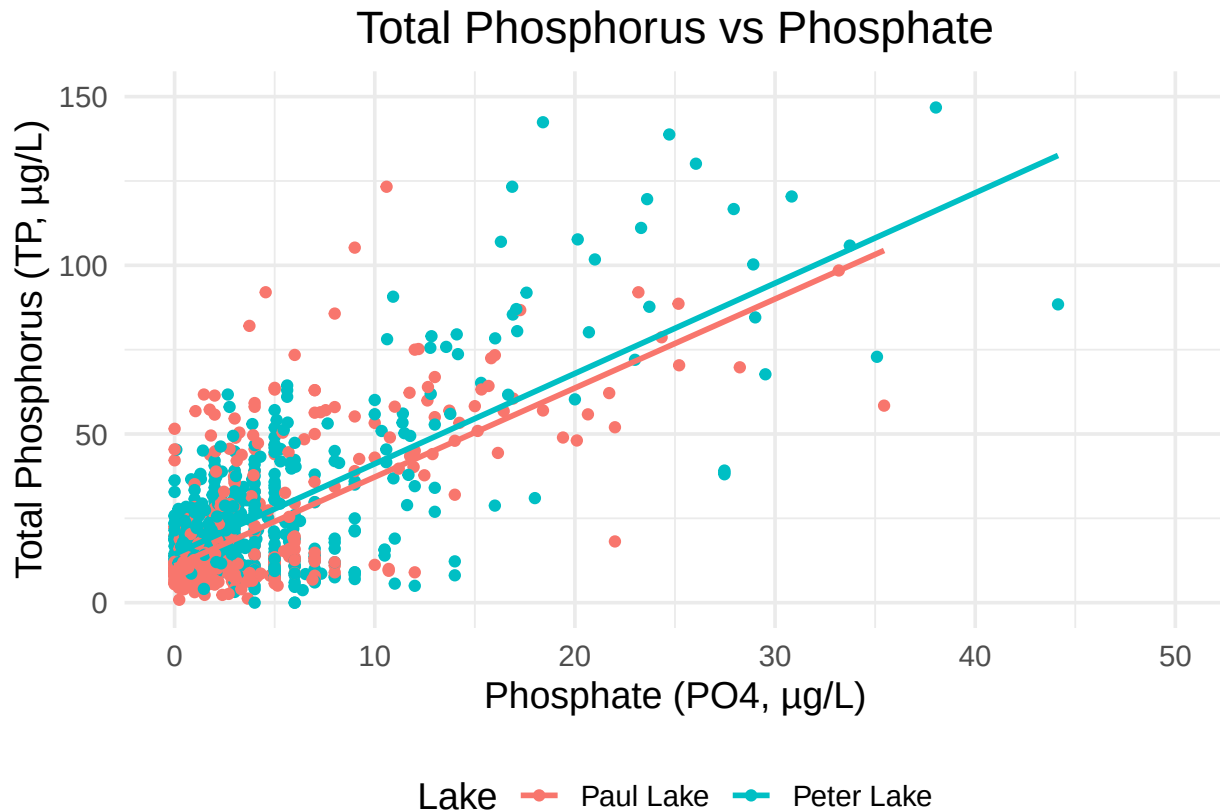
  color = "Lake"
) +
theme_minimal(base_size = 14) +
theme(
  plot.title = element_text(hjust = 0.5),
  legend.position = "bottom"
)

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
## Warning: Removed 21948 rows containing non-finite outside the scale range
## ('stat_smooth()').
```

```
## Warning: Removed 21948 rows containing missing values or values outside the scale range
## ('geom_point()').
```



5. [NTL-LTER] Make three separate boxplots of (a) temperature, (b) TP, and (c) TN, with month as the x axis and lake as a color aesthetic. Then, create a cowplot that combines the three graphs. Make sure that only one legend is present and that graph axes are aligned. Show all months, even those where no data was collected.

Tips: * Recall the discussion on factors in the lab section as it may be helpful here. * Setting an axis text in your theme to `element_blank()` removes the axis text (useful when multiple, aligned plots use the same axis values) * Setting a legend's position to "none" will remove the legend from a plot. * Individual plots can have different relative sizes when combined using `cowplot`.

```

#5

#loading libraries
library(ggplot2)
library(cowplot)
library(dplyr)

#converting sampleddate to date format
lake_nutrients$sampleddate <- as.Date(lake_nutrients$sampleddate)

#creating month column from sampleddate & converting to factor
lake_nutrients$month <- factor(months(lake_nutrients$sampleddate, abbreviate = TRUE),
                               levels = month.abb)

#plotting temp
plot_temp <- ggplot(lake_nutrients, aes(x = month, y = temperature_C, fill = lakename)) +
  geom_boxplot() +
  labs(title = "Temperature", y = "Temperature (°C)") +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "none",
    axis.text.x = element_blank(),
    axis.title.x = element_blank()
  )

#plotting total phosphorus
plot_tp <- ggplot(lake_nutrients, aes(x = month, y = tp_ug, fill = lakename)) +
  geom_boxplot() +
  labs(title = "Total Phosphorus", y = "TP (µg/L)") +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "none",
    axis.text.x = element_blank(),
    axis.title.x = element_blank()
  )

#plotting total nitrogen (keeping legend & x-axis for bottom plot)
plot_tn <- ggplot(lake_nutrients, aes(x = month, y = tn_ug, fill = lakename)) +
  geom_boxplot() +
  labs(title = "Total Nitrogen", x = "Month", y = "TN (µg/L)", fill = "Lake") +
  theme_minimal(base_size = 14) +
  theme(
    legend.position = "bottom"
  )

#combing the 3 plots using cowplot
cowplot::plot_grid(plot_temp, plot_tp, plot_tn,
                    ncol = 1,
                    align = "v",
                    rel_heights = c(1, 1, 1.2))

```

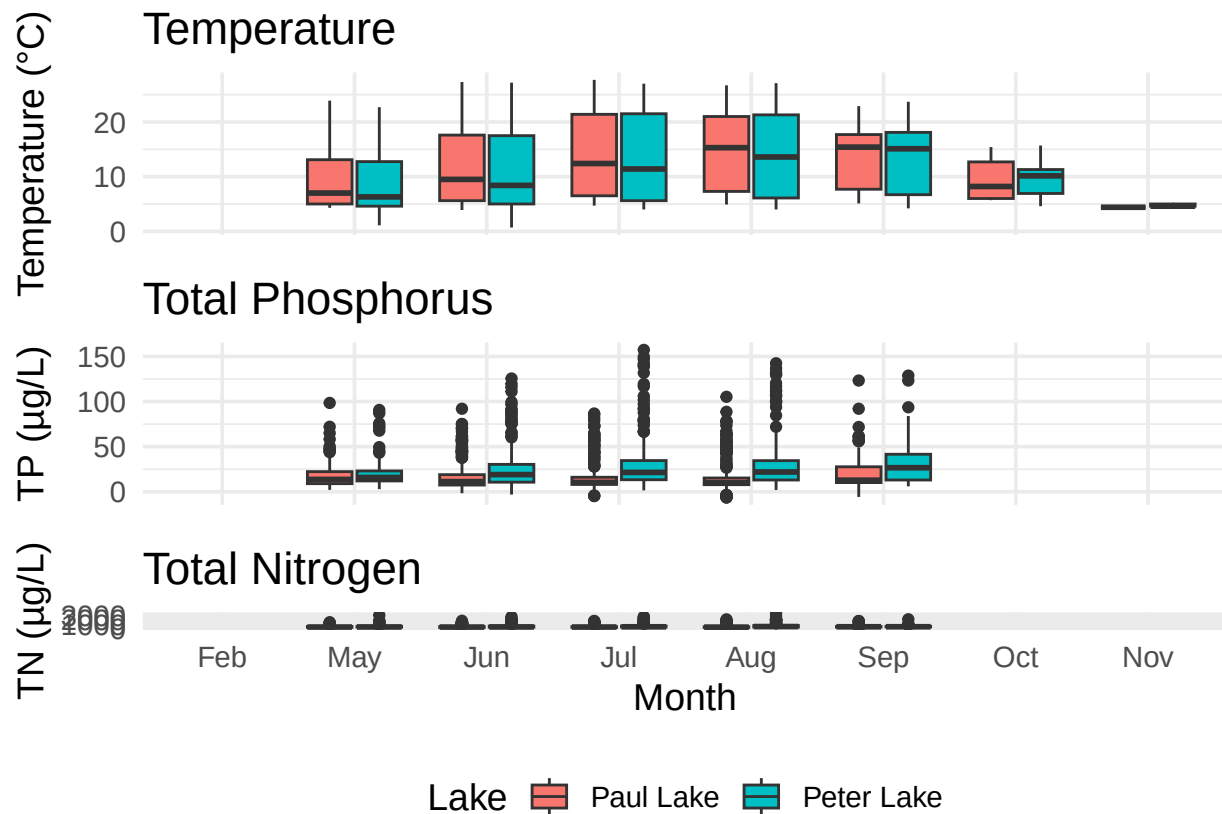
```

## Warning: Removed 3566 rows containing non-finite outside the scale range
## ('stat_boxplot()').

```

```
## Warning: Removed 20729 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

```
## Warning: Removed 21583 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```



Question: What do you observe about the variables of interest over seasons and between lakes?

Answer: Temperature shows clear seasonal variation, with slightly higher temperatures in Paul Lake compared to Peter Lake. Both Total Phosphorus (TP) and Total Nitrogen (TN) remain relatively stable across seasons, but Paul Lake shows slightly higher and more variable TN levels. While both lakes have consistent nutrient levels over the months, Paul Lake appears more sensitive to changes, especially in nitrogen concentrations.

6. [Niwot Ridge] Plot a subset of the litter dataset by displaying only the “Needles” functional group. Plot the dry mass of needle litter by date and separate by NLCD class with a color aesthetic. (no need to adjust the name of each land use)
7. [Niwot Ridge] Now, plot the same plot but with NLCD classes separated into three facets rather than separated by color.

#6

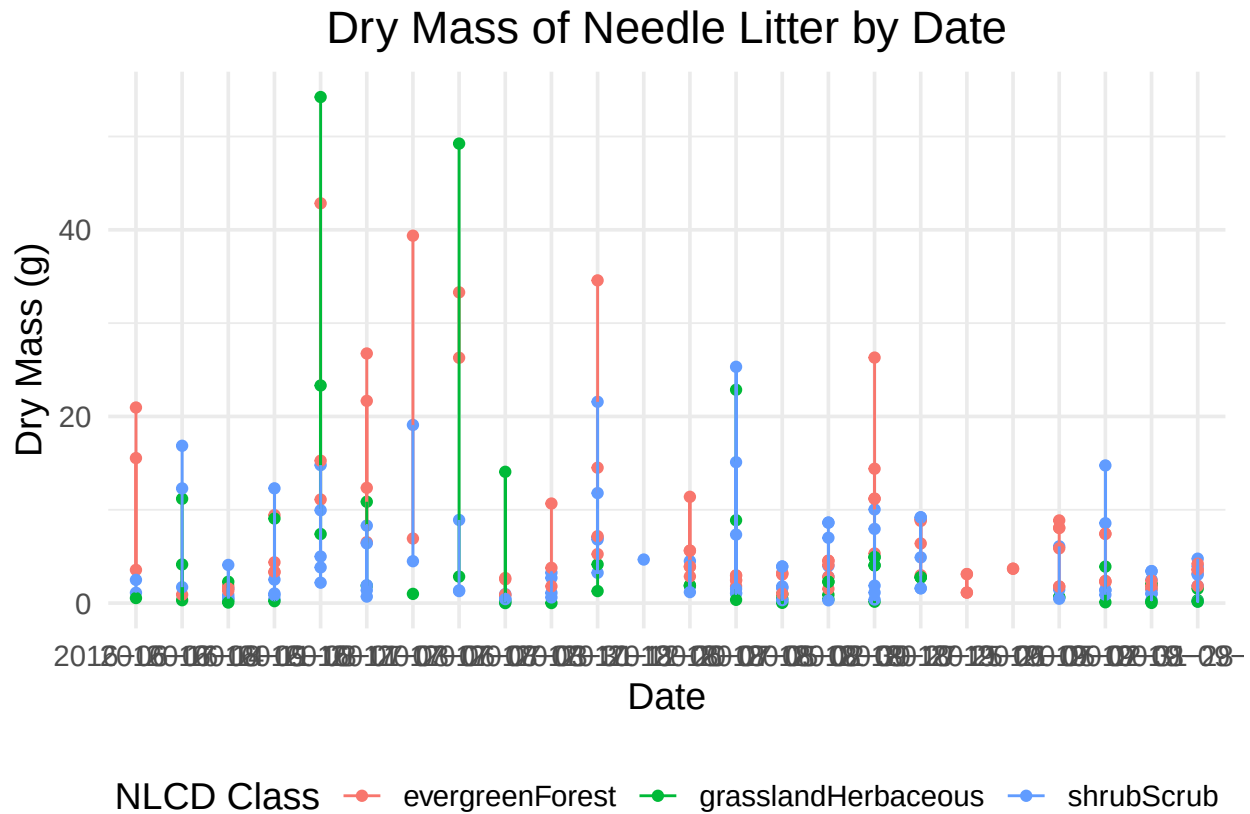
```
#checking the col names
colnames(litter)
```

```
## [1] "plotID"          "trapID"          "collectDate"     "functionalGroup"
## [5] "dryMass"         "qaDryMass"       "subplotID"       "decimalLatitude"
## [9] "decimalLongitude" "elevation"       "nlcdClass"       "plotType"
## [13] "geodeticDatum"
```

```
#loading necessary libraries
library(ggplot2)
library(dplyr)

#filtering for 'Needles' func grp
litter_needles <- litter %>%
  filter(functionalGroup == "Needles")

#plotting the dry mass of needle litter by date + separating by nlcd class +
#scattering plot for dryMass by collectDate + adding lines to connect points
ggplot(litter_needles, aes(x = collectDate, y = dryMass, color = nlcdClass)) +
  geom_point() + #
  geom_line() +
  labs(
    title = "Dry Mass of Needle Litter by Date",
    x = "Date",
    y = "Dry Mass (g)",
    color = "NLCD Class"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5),
    legend.position = "bottom"
  )
```

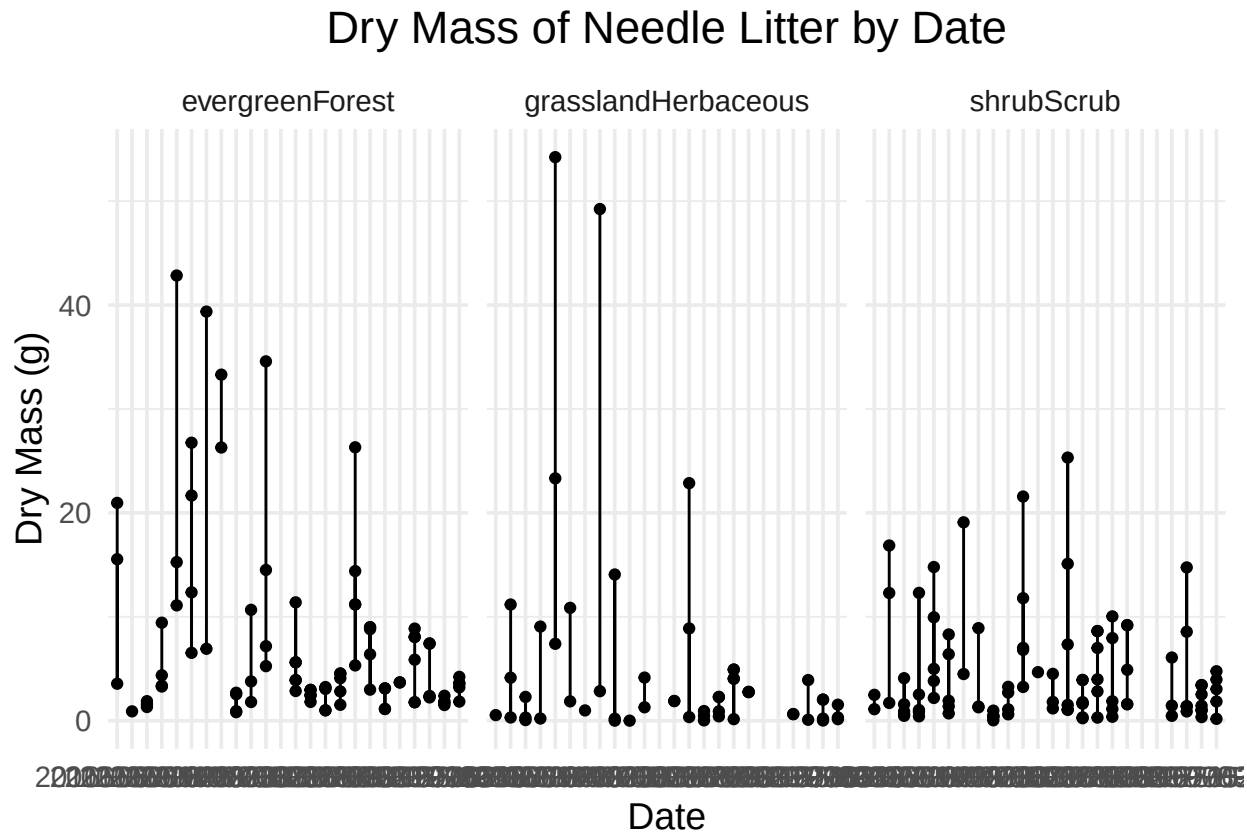



```
#7

# filtering for 'Needles' func grp
litter_needles <- litter %>%
  filter(functionalGroup == "Needles")

#plotting the dry mass of needle litter by date + faceting by nlcd class
ggplot(litter_needles, aes(x = collectDate, y = dryMass)) +
  geom_point() +
  geom_line() +
  facet_wrap(~ nlcdClass) +
  labs(
    title = "Dry Mass of Needle Litter by Date",
    x = "Date",
    y = "Dry Mass (g)"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5)
  )

```



Question: Which of these plots (6 vs. 7) do you think is more effective, and why?

Answer: Plot 7 with facets is more effective because it's easier to see the trends over time for each land cover type without the lines and points getting jumbled together. Each panel gives a clearer view of the seasonal patterns, making it simpler to spot what's happening in each NLCD class individually.